Standard Library Support for Void

ISO/IEC JTC1 SC22 WG21
Document Number: P0242R0
Audience: Library Evolution Working Group
Matt Calabrese (metaprogrammingtheworld@gmail.com)
2016-02-11

Abstract

This paper assumes the updated `void` type described in P0146R1[1] and provides operator overloads for standard input and output with `void` such that generic code which deals with input and output will be compatible with `void` types. In addition to that, it also declares a specialization of `std::hash` for `void`.

Motivation

With the updated `void` type described in P0146R1[1], `void` may be instantiated and used as an object. Because of this, users may expect the standard library to support the type in similar ways that it does for many other built-in types, such as arithmetic types. One place where support may be desirable is with iostreams, allowing an instance of `void` to be input or output without users having to explicitly provide their own operator overloads. The desire for such support can come up in generic code that outputs the result of a function call using a standard output stream, for example. It would be useful if such generic code would work with `void` return types, and without users having to handle the type in a special manner. Similar concerns come up regarding function templates that hash the value of a dependent type.

Alternatives

Undeclared Insertion and Extraction for `void`

An alternative to providing operator overloads for insertion and extraction of `void` could be to simply leave these operators undeclared in the standard, allowing users to define their own, if needed. This has some problems:

- The need for developers to do this is inconsistent with many other built-in types.
- If multiple projects (i.e. libraries) declare their own overloads for `void`, they will encounter multiple definitions or ODR violations when used in the same project.
- In practice, users would have to put the overloads in the global namespace in order for them to be found from other namespaces, and those declarations would have to appear before the definition of any generic code that uses them, even if the template is instantiated after the overload is introduced, which is a very subtle detail for developers to understand (if unclear, ADL will not find such an overload in the global namespace if it is first declared between the definition of the generic function that makes the unqualified call and its instantiation, unless a separate user-defined type brings in the global namespace, since fundamental types do not introduce ADL, nor do they necessarily bring in the global namespace as an associated namespace in the event that ADL does occur).

Non-Empty Input and Output for `void`

Though it is recommended that operators for input and output should be provided for `void`, what the precise behavior of those operators should be is somewhat subjective. This proposal recommends treating input and output of `void` as formatted input and output functions that effectively do nothing other than return a reference to the passed-in stream. Alternatively, it could be specified that output and input deal with a consistent string, such as "void," "{}," of "0", but this is unnecessary and introduces a cost to the input and output of `void` that does not need to exist.

Undeclared `std::hash` support for `void`

An alternative to providing a `std::hash` specialization for `void` could be to leave such a specialization undeclared in the standard, allowing users to define their own, if needed. This has some problems:

- The need for developers to do this is inconsistent with many other built-in types.
- If multiple projects (i.e. libraries) declare their own specializations for `void`, they will encounter multiple definitions or ODR violations when used in the same project.

Proposed Solution

This proposal suggests adding << and >> operator overloads for `std::basic_ostream` and `std::basic_istream` respectively, along with a declared specialization of `std::hash` for `void`.

Add a `void` specialization of `std::hash` to the synopsis in **§20.9.12 [unord.hash]** paragraph 1:

```
template <> struct hash<void>;
```

Add paragraphs to the end of **§27.7.2.2.3 [istream::extractors]**:

```
template<class charT, class traits>
basic_istream<charT,traits>& operator>>(basic_istream<charT,traits>& in,
                                        void& v);
Effects: Behaves like a formatted input function ( 27.7.2.2.1) of in.
Returns: *this.
```

Add section **§27.7.3.6.5 Void inserter function templates [ostream.inserters.void]**:

```
template<class charT, class traits>
  basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>& out,
                                          void v);
Effects: Behaves as a formatted output function ( 27.7.3.6.1) of out.
Returns: *this.
```

References

[1] Matt Calabrese: "Regular Void" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0146r1.html